

MAE 6291

Internet of Things for Engineers

Prof. Kartik Bulusu, MAE Dept.

Week 5 [02/18/2026]

- Layers in IoT systems - 3 level layer model
- Recap The Linux Terminal
- Recap: Setting up a Python virtual environment [venv] - Sandboxing your code
- MQTT your LED - Your first IoT program [Graded Lab Activity]
- Raspberry Pi messaging with Python scripts [Graded Lab Activity]

`git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git`



School of Engineering
& Applied Science

Spring 2026

THE GEORGE WASHINGTON UNIVERSITY

Photo: Kartik Bulusu

Content Attribution & Sources

This course material incorporates both original content and supplementary visual aids:

- **Images with references:** Properly cited in slide captions, figure legends, or in-text attributions
- **Unreferenced images:** Sourced from Perplexity.io and used for illustrative purposes in an educational context
- **Fair use:** All materials are used in compliance with educational fair use guidelines



Resetting the course for the next 9 weeks:

- Topics to be covered
- Weekly deliverables



Hardware:

1. ESP32
2. Cameras
3. SenseHat

Mathematics:

1. Basics of matrices
2. Applications of matrices: filters
3. Basics of Signal processing

App-development:

1. Flask
2. Micropython
3. Flask_Restful

Edge computing on the Pi:

Mathematics + Python + Signal processing

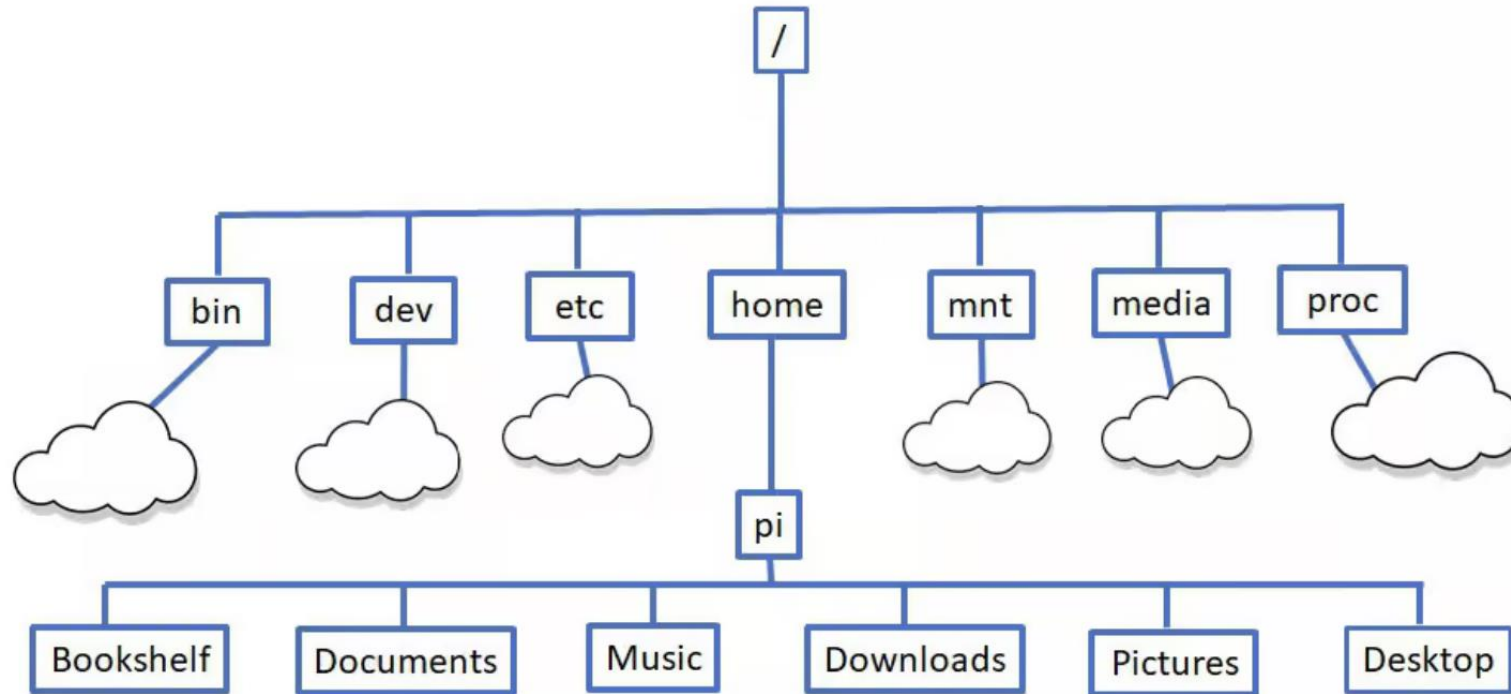
Expectations on student deliverables:

1. Midterm project demo
2. Midterm project presentation
3. Midterm project report in a conference-style template
4. Midterm project: Two min video submission
5. Weekly coding HW
6. Weekly Quizzes
7. Graduate students: Paper reviews
8. Final project proposal
9. Final project presentation
10. Final project demo
11. Final report in a conference-style template



The Linux Terminal





Explore files and folders

Command	Description
pwd	Show the current directory you are in
ls	List files in the current directory; add -l or -a for more detail or to see hidden files
cd /path/goes/here	Change directory (for example cd /home/pi/Documents)
mkdir myfolder	Create a new directory named myfolder
cp file1 file2	Copy a file; cp -r dir1 dir2 copies a whole directory
mv old new	Move or rename files/directories
rm file	Delete a file; use rm -r folder carefully to remove a directory



Helpful navigation and history tips

Command or Keyboard Entry	Description
clear	Clear the terminal screen.
history	Show a numbered list of past commands; run one again with !number.
Up/Down arrow keys	Scroll through previously used commands to run or edit them.



View and edit files

Command	Description
nano file.txt	Open a simple text editor in the terminal to create or edit a file
less file.txt	View a long file one page at a time (use q to quit)
cat file.txt	Print the contents of a text file to the screen



System information and status

Command	Description
hostname -I	Show the Pi's IP address
uname -a	Show Linux kernel and system details
df -h	Show disk space usage in human-readable form
free -h	Show memory (RAM) usage
uptime	See how long the Pi has been running
top or htop	Monitor running processes and CPU usage (press q to exit top).



IP address of your RPi connected to the network using WiFi

ifconfig -a

```
pi@raspberrypi017: ~  
File Edit Tabs Help  
pi@raspberrypi017:~$ ifconfig -a  
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255  
    ether ea:e0:b9:19:85:8f txqueuelen 0 (Ethernet)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 169.254.247.201 netmask 255.255.0.0 broadcast 169.254.255.255  
    inet6 fe80::21e4:d41c:791a:a9d7 prefixlen 64 scopeid 0x20<link>  
    ether dc:a6:32:98:38:24 txqueuelen 1000 (Ethernet)  
    RX packets 1874 bytes 166519 (162.6 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 1723 bytes 1258105 (1.1 MiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 53 bytes 5297 (5.1 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 53 bytes 5297 (5.1 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.198.24.23 netmask 255.255.252.0 broadcast 10.198.27.255  
    inet6 fe80::3d8a:ccb:6a3a:2f0 prefixlen 64 scopeid 0x20<link>  
    ether dc:a6:32:98:38:25 txqueuelen 1000 (Ethernet)  
    RX packets 12 bytes 1379 (1.3 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 67 bytes 8066 (7.8 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
pi@raspberrypi017:~$
```



Building up the IoT Architecture and Ecosystem

Icon Sources:

sensor by Carolina Cani; sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo:

<https://thenounproject.com/browse/icons/term/sensor/>

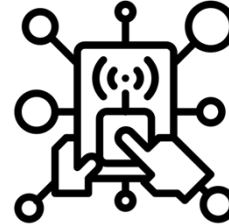
fire sensor by LAFS: <https://thenounproject.com/browse/icons/term/fire-sensor/>

Ultrasound by Shocho: <https://thenounproject.com/browse/icons/term/ultrasound/>

Network by Solikin; Network by Tippawan: <https://thenounproject.com/browse/icons/term/network/>

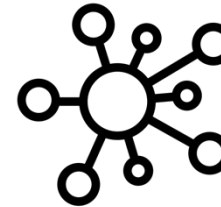
application by Chaowalit Koetchuea: <https://thenounproject.com/browse/icons/term/application/>

Information-layer



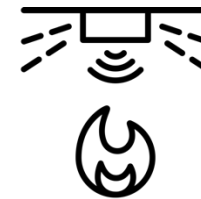
Data

Communication-layer



Connectivity

Sensor-layer



Things



The 3-Layer IoT Architecture

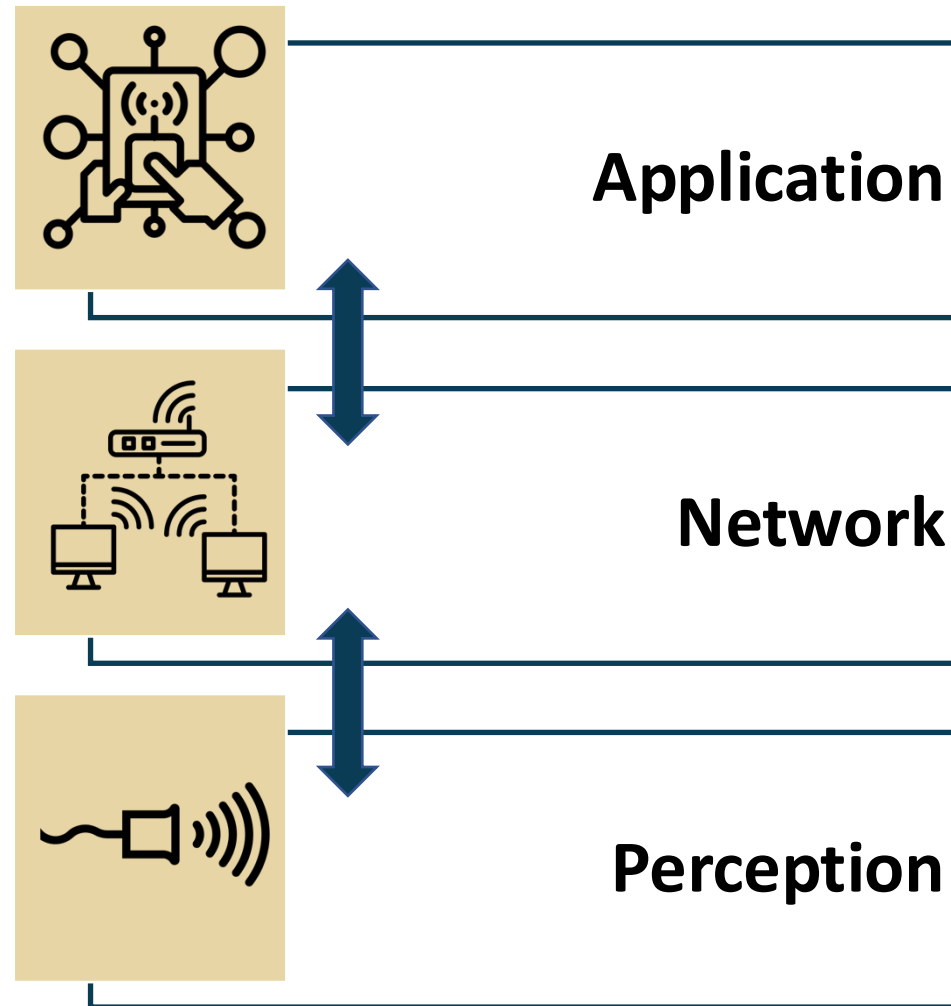
Icon Sources:

sensor by Carolina Cani, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo:

<https://thenounproject.com/browse/icons/term/sensor>

wifi network by Matthias Hartmann: <https://thenounproject.com/browse/icons/term/wifi-network/>

application by Chaowalit Koetchuea: <https://thenounproject.com/browse/icons/term/application/>



Skeleton of the updated Python program written for Raspberry Pi

```
import library1 as name1  
import RPi.GPIO as GPIO  
import time
```

Import libraries that are relevant for interaction with the Raspberry Pi hardware such as GPIO pins, camera ports etc.

```
GPIO.setmode(GPIO.BOARD)
```

```
GPIO.setup(12, GPIO.OUT)
```

Set up GPIO pins as data outputs or inputs for sensors and actuators

```
def function_name(arguments):  
    statement  
    statement  
    ...  
    return value
```

Create user-defined functions to modularize your code and make it easy to work.

Create user-defined functions to release the GPIO pins

```
if __name__ == "__main__":  
    try:  
        function_name1()  
    except KeyboardInterrupt:  
        function_name2()
```

Create entry point into the program and pull all functions in

Create a keyboard-interrupt exception clause



Setting up a Python virtual environment - “Sandbox” your Python projects

“Unwearyingly working and making changes to the global Python environment can break Python-based system-level tools, and remedying the situation can become a major headache.”

Gary Smart, Practical Python Programming for IoT

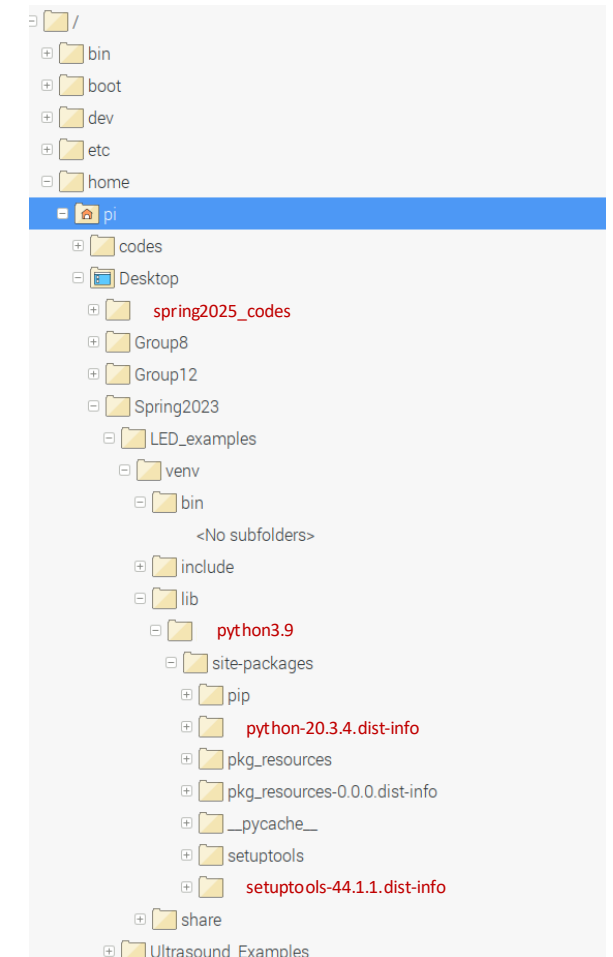


Create python virtual environments

- To “sandbox” our Python projects
- Avoid adverse interference with
 - system-level Python utilities
 - or other Python projects.

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
pi@raspberrypi:~ $ cd Desktop/spring2025_codes/LED_examples/
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python -m venv venv
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ source venv/bin/activate
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ which python
/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python --version
Python 3.9.2
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ deactivate
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

Anatomy of the python virtual environments



Activate python-venv & Install pip

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help

pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ source venv/bin/activate
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip install --upgrade pip
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Requirement already satisfied: pip in ./venv/lib/python3.9/site-packages (20.3.4)
Collecting pip
  Downloading https://www.piwheels.org/simple/pip/pip-25.0-py3-none-any.whl (1.8 MB)
    |██████████| 1.8 MB 420 kB/s
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 20.3.4
    Uninstalling pip-20.3.4:
      Successfully uninstalled pip-20.3.4
  Successfully installed pip-25.0
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip list
Package            Version
-----
pip                25.0
pkg_resources      0.0.0
setuptools         44.1.1
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

Review of the code provided to you

```
Thonny - /home/pi/Desktop/spring2025_co...LED_examples/gpio_pkg_check.py @ 32:75
File Edit View Run Tools Help

gpio_pkg_check.py x
1 """
2 File: chapter01/gpio_pkg_check.py
3 Source: Practical Python Programming for IoT
4 Author: Gary Smart
5
6 This Python script checks for the availability of various Python GPIO Library Packages f
7 It does this by attempting to import the Python package. If the package import is succes
8 we report the package as Available, and if the import (or import initialization) fails f
9 we report the package as Unavailable.
10
11 Dependencies:
12 pip3 install gpiozero pigpio
13
14 Built and tested with Python 3.7 on Raspberry Pi 4 Model B
15 """
16
17 try:
18     import gpiozero
19     print('GPIOZero    Available')
20 except:
21     print('GPIOZero    Unavailable. Install with "pip install gpiozero"')
22
23 try:
24     import pigpio
25     print('PiGPIO      Available')
26 except:
27     print('PiGPIO      Unavailable. Install with "pip install pigpio"')
28
29 try:
30     import RPi.GPIO
31     print('RPi.GPIO    Available')
32 except:
```

Variables x

Name	Value
------	-------

Assistant x

Shell x

Python 3.9.2 (/usr/bin/python3)

>>>

Local Python 3 • /usr/bin/python3



Install GPIO packages using the code provided.

Note: You are still in python-venv

Checks and balances:
pip list
pip freeze > requirements

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python3 pkg_check.py
GPIOZero Unavailable. Install with "pip install gpiozero"
PiGPIO Unavailable. Install with "pip install pigpio"
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip install gpiozero RPi.GPIO
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting gpiozero
  Downloading https://www.piwheels.org/simple/gpiozero/gpiozero-2.0.1-py3-none-any.whl (150 kB)
Collecting pigpio
  Downloading https://www.piwheels.org/simple/pigpio/pigpio-1.78-py2.py3-none-any.whl (39 kB)
Collecting RPi.GPIO
  Downloading https://www.piwheels.org/simple/rpi-gpio/RPi.GPIO-0.7.1-cp39-cp39-linux_armv7l.whl (66 kB)
Collecting colorzero (from gpiozero)
  Downloading https://www.piwheels.org/simple/colorzero/colorzero-2.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources~=5.0 (from gpiozero)
  Downloading https://www.piwheels.org/simple/importlib-resources/importlib_resources-5.13.0-py3-none-any.whl (32 kB)
Collecting importlib-metadata~=4.6 (from gpiozero)
  Downloading https://www.piwheels.org/simple/importlib-metadata/importlib_metadata-4.13.0-py3-none-any.whl (23 kB)
Collecting zipp>=0.5 (from importlib-metadata~=4.6->gpiozero)
  Downloading https://www.piwheels.org/simple/zipp/zipp-3.21.0-py3-none-any.whl (9.6 kB)
Requirement already satisfied: setuptools in ./venv/lib/python3.9/site-packages (from colorzero->gpiozero) (44.1.1)
Installing collected packages: RPi.GPIO, pigpio, zipp, colorzero, importlib-resources, importlib-metadata, gpiozero
Successfully installed RPi.GPIO-0.7.1 colorzero-2.0 gpiozero-2.0.1 importlib-metadata-4.13.0 importlib-resources-5.13.0 pigpio-1.78 zipp-3.21.0
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip list
Package Version
-----
colorzero 2.0
gpiozero 2.0.1
importlib-metadata 4.13.0
importlib-resources 5.13.0
pigpio 1.78
pip 25.0
pkg_resources 0.0.0
RPi.GPIO 0.7.1
setuptools 44.1.1
zipp 3.21.0
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python3 pkg_check.py
GPIOZero Available
PiGPIO Available
RPi.GPIO Available
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip freeze > requirements
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

pip stands for “Python installs packages”.

pip queries the Python Package Index, or simply PyPi

Web source for PyPi: <https://pypi.org>



Run the python codes
Blink your LEDs
Note: You are still in python-venv

deactivate your venv

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ which python
/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python --version
Python 3.9.2
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python gpio.py check.py
GPIOZero Available
PiGPIO Available
RPi.GPIO Available
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python LED.py
0
1
2
3
4
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ clear
```

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ deactivate
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```



Running a python script at boot cron, the UNIX scheduler

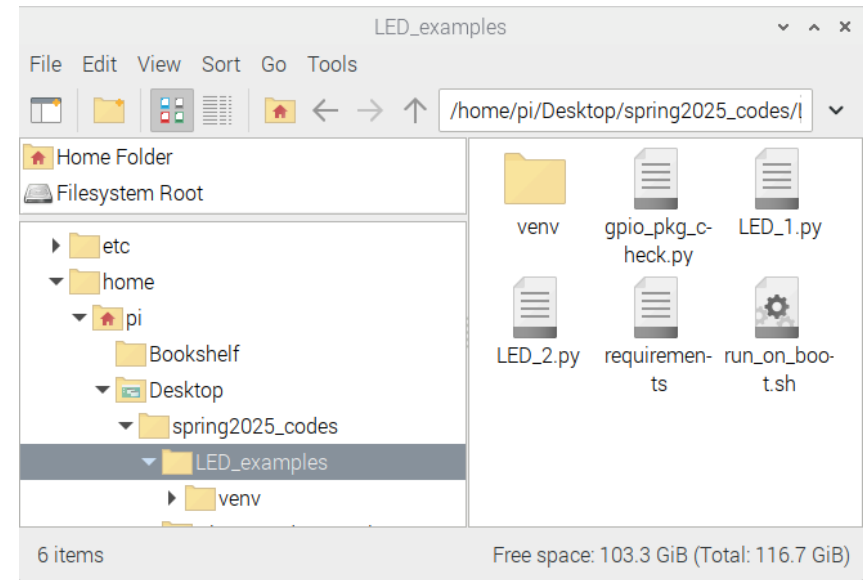
Awesome IoT project will run automatically every time you start your Raspberry Pi



Review run_on_boot.sh

Note: You are no longer in python-venv

Let's make sure we have a file-folder structure that looks some what like this



Let's open the *run_on_boot.sh* on a terminal to review the script



Absolute paths to

1. Python interpreter in venv
2. Python script
3. Locate the LOG-file



The last two lines tie everything together

1. echo [updates LOG-file with string in quotes]
2. Next line runs py-file and logs output on boot
3. 2>&1 [logs any additional errors during run]



```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ cat run_on_boot.sh
#!/bin/bash
#File: LED_examples/run_on_boot.sh

# Absolute path to Virtual Environment python interpreter
PYTHON=/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python

# Absolute path to Python script
SCRIPT=/home/pi/Desktop/spring2025_codes/LED_examples/LED_1.py

# Absolute path to output log file
LOG=/home/pi/Desktop/spring2025_codes/LED_examples/LED_1.log

echo -e "\n##### STARTUP $(date) #####\n" >> $LOG

$PYTHON $SCRIPT >> $LOG 2>&1
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

chmod u+x run_on_boot.sh
*current **U**ser, make the file e**X**ecutable*

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ cat run_on_boot.sh
#!/bin/bash
#File: LED_examples/run_on_boot.sh

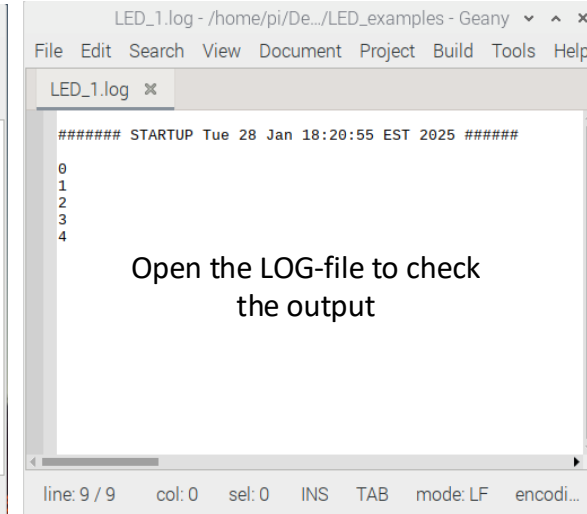
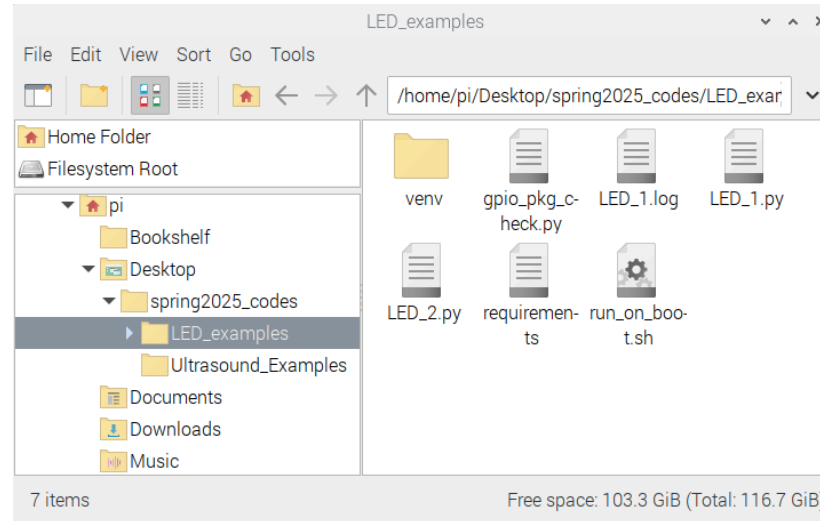
# Absolute path to Virtual Environment python interpreter
PYTHON=/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python

# Absolute path to Python script
SCRIPT=/home/pi/Desktop/spring2025_codes/LED_examples/LED_1.py

# Absolute path to output log file
LOG=/home/pi/Desktop/spring2025_codes/LED_examples/LED_1.log

echo -e "\n##### STARTUP $(date) #####\n" >> $LOG

$PYTHON $SCRIPT >> $LOG 2>&1
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ chmod u+x run_on_boot.sh
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```



./run_on_boot.sh

```
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ ./run_on_boot.sh
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```



crontab -e

This is a list of editors used in terminal with Linux or Unix OS

I recommend you choose “nano” or the default Option-1 and hit “Enter”.

“vim” and “ed” are also popular with advanced or legacy users

“code” will open up VS Code that slows down the Raspberry Pi OS

```
$ crontab -e
```

*no crontab for pi - using an empty one
Select an editor. To change later, run 'select-editor'.*

- -- --> {
1. /bin/nano <---- easiest
 2. /usr/bin/vim.tiny
 3. /usr/bin/code
 4. /bin/ed

Choose 1-4 [1]: 1



crontab -e

"Do not forget the & character at the end of the line. This makes sure the script runs in the background."

Gary Smart, Practical Python Programming for IoT

```
$ crontab -e
```

```
@reboot /home/pi/Desktop/spring2026_codes/LED_examples/run_on_boot.sh &
```

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
GNU nano 5.4 /tmp/crontab.MInc7f/crontab *
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
@reboot /home/pi/Desktop/spring2026_codes/LED_examples/run_on_boot.sh &
```

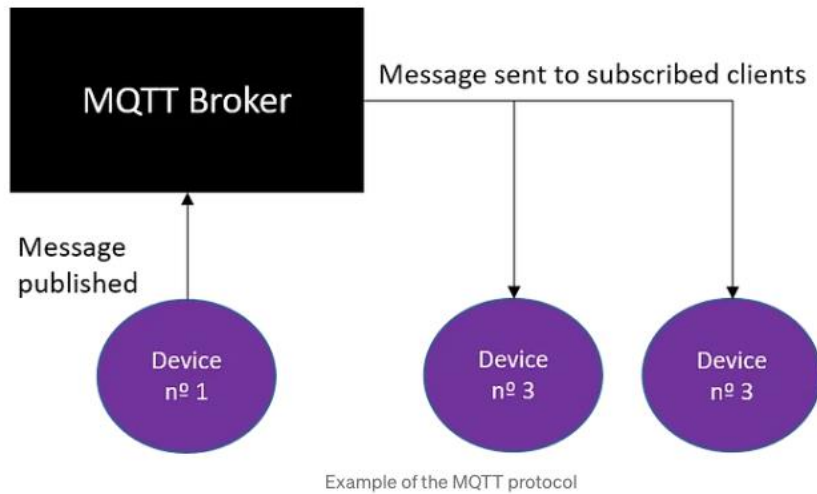
Try manually `./run_on_boot.sh`
then
`sudo reboot`

```
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ ./run_on_boot.sh
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ sudo reboot
```

To stop a cron job you can type this on the terminal:
`sudo service cron stop`



MQTT paradigm



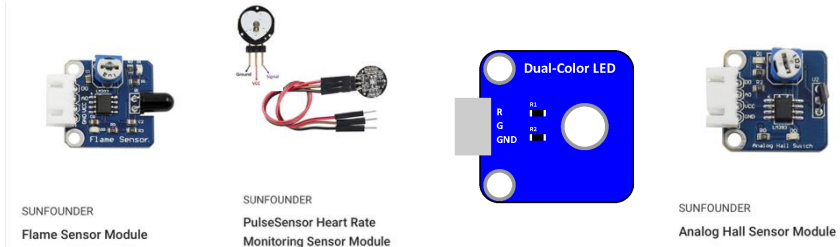
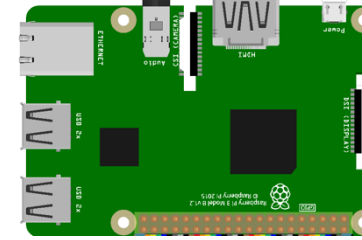
Hardware

Broker

- The broker is the server
- It distributes the information to the interested devices connected to the server.

Client

- The device that connects to broker to send or receive information.



Messaging

Topic

- The name that the message is about.
- Clients publish, subscribe, or do both to a topic.

Subscribe

- Clients tell the broker which topic(s) they're interested in.

Publish

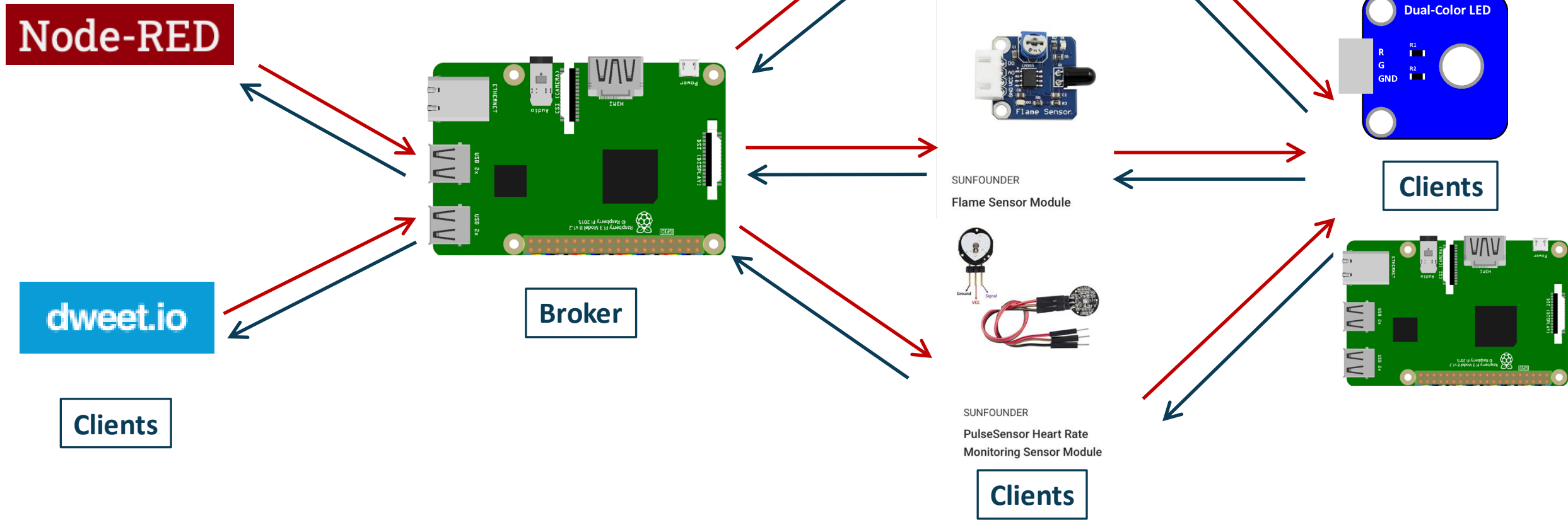
- Clients that send information to the broker to distribute to interested clients based on the topic name.

QoS

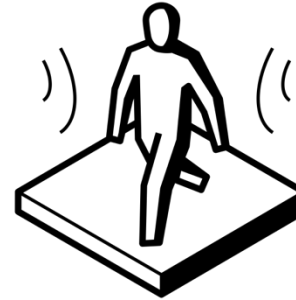
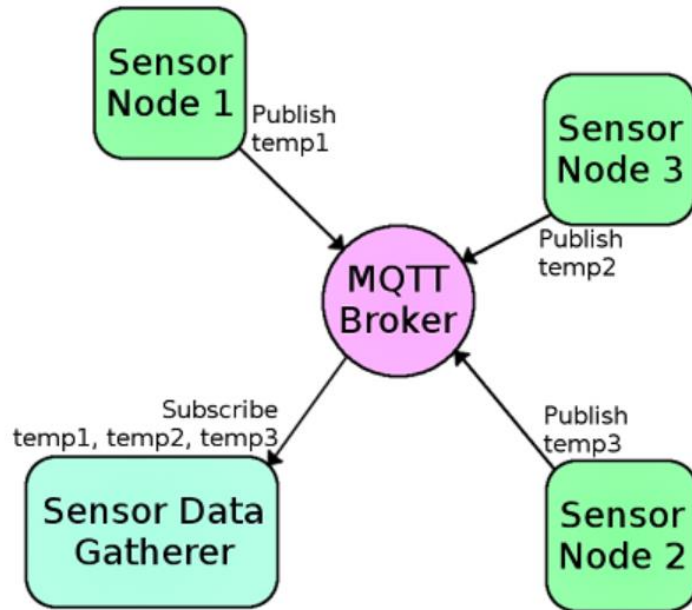
- Quality of Service to the broker
- Integer value ranging from 0-2.

Source: <https://andre-benevides.medium.com/introduction-to-mqtt-and-configuration-of-a-mosquitto-broker-f0f7a7738bc8>
<https://learn.sparkfun.com/tutorials/introduction-to-mqtt#the-basics>

Practical view of MQTT in IoT applications



MQTT paradigm

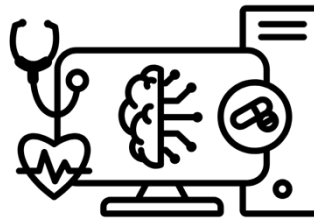


Created by Tomas Knopp
from Noun Project



Created by Carolina Cani
from Noun Project

Common usages



Created by Sinta Maulana
from Noun Project



Created by Rolas Design
from Noun Project

Source:
<https://learn.sparkfun.com/tutorials/introduction-to-mqtt#the-basics>
<https://andre-benevides.medium.com/introduction-to-mqtt-and-configuration-of-a-mosquitto-broker-f0f7a7738bc8>

Health monitoring by Sinta Maulana from [Noun Project](#) (CC BY 3.0)
motion sensor by Tomas Knopp from [Noun Project](#) (CC BY 3.0)
fire sensor by Carolina Cani from [Noun Project](#) (CC BY 3.0)
chat by Rolas Design from [Noun Project](#) (CC BY 3.0)



Eclipse Mosquitto - An open source MQTT broker



Eclipse Mosquitto provides a lightweight server implementation of the MQTT protocol that is suitable for all situations from full power machines to embedded and low power machines.

Sensors and actuators, which are often the sources and destinations of MQTT messages, can be very small and lacking in power. This also applies to the embedded machines to which they are connected, which is where Mosquitto could be run.



Step-1: Eclipse Mosquitto - An open source MQTT broker

sudo apt-get update

sudo apt-get upgrade

sudo apt install mosquitto

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ sudo apt install mosquitto
```

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ sudo apt install mosquitto  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
The following additional packages will be installed:  
  libev4 libwebsockets8  
Suggested packages:  
  apparmor  
The following NEW packages will be installed:  
  libev4 libwebsockets8 mosquitto  
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.  
Need to get 241 kB of archives.  
After this operation, 543 kB of additional disk space will be used.  
Do you want to continue? [Y/n] y
```

```
File Edit Tabs Help  
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.  
Need to get 241 kB of archives.  
After this operation, 543 kB of additional disk space will be used.  
Do you want to continue? [Y/n] y  
Get:1 http://mirror.pit.teraswitch.com/raspbian/raspbian stretch/main armhf libev4 armhf 1:4.22-1 [34.0 kB]  
Get:2 http://mirror.us.leaseweb.net/raspbian/raspbian stretch/main armhf mosquitto armhf 1.4.10-3+deb9u5 [122 kB]  
Get:3 http://archive.raspberrypi.org/debian stretch/main armhf libwebsockets8 armhf 2.0.3-2+b1-rpt1 [85.2 kB]  
Fetched 241 kB in 5s (44.1 kB/s)  
Selecting previously unselected package libev4.  
(Reading database ... 97725 files and directories currently installed.)  
Unpacking libev4 (1:4.22-1) ...  
Selecting previously unselected package libwebsockets8:armhf.  
Unpacking libwebsockets8:armhf (2.0.3-2+b1-rpt1) ...  
Selecting previously unselected package mosquitto.  
Unpacking mosquitto (1.4.10-3+deb9u5) ...  
Setting up libev4 (1:4.22-1) ...  
Processing triggers for libc-bin (2.24-11+deb9u4) ...  
Setting up mosquitto (1.4.10-3+deb9u5) ...  
Processing triggers for systemd (232-25+deb9u14) ...
```



Step-2: restart Mosquitto

`sudo /etc/init.d/mosquitto restart`

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ sudo /etc/init.d/mosquitto restart  
[ ok ] Restarting mosquitto (via systemctl): mosquitto.service.  
pi@raspberrypi:~ $
```

Step-3: Get your IP address

`ifconfig`

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ ifconfig  
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
    ether b8:27:eb:3d:a8:1c txqueuelen 1000 (Ethernet)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 73 bytes 3847 (3.7 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 73 bytes 3847 (3.7 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.1.216 netmask 255.255.255.0 broadcast 192.168.1.255  
    inet6 2600:4040:2db8:400:702:5225:99c5:a013 prefixlen 64 scopeid 0x0<g  
    loba>  
    inet6 fe80::17d0:331:ba10:cde3 prefixlen 64 scopeid 0x20<link>  
    ether b8:27:eb:68:fd:49 txqueuelen 1000 (Ethernet)  
    RX packets 868 bytes 857162 (837.0 KiB)
```

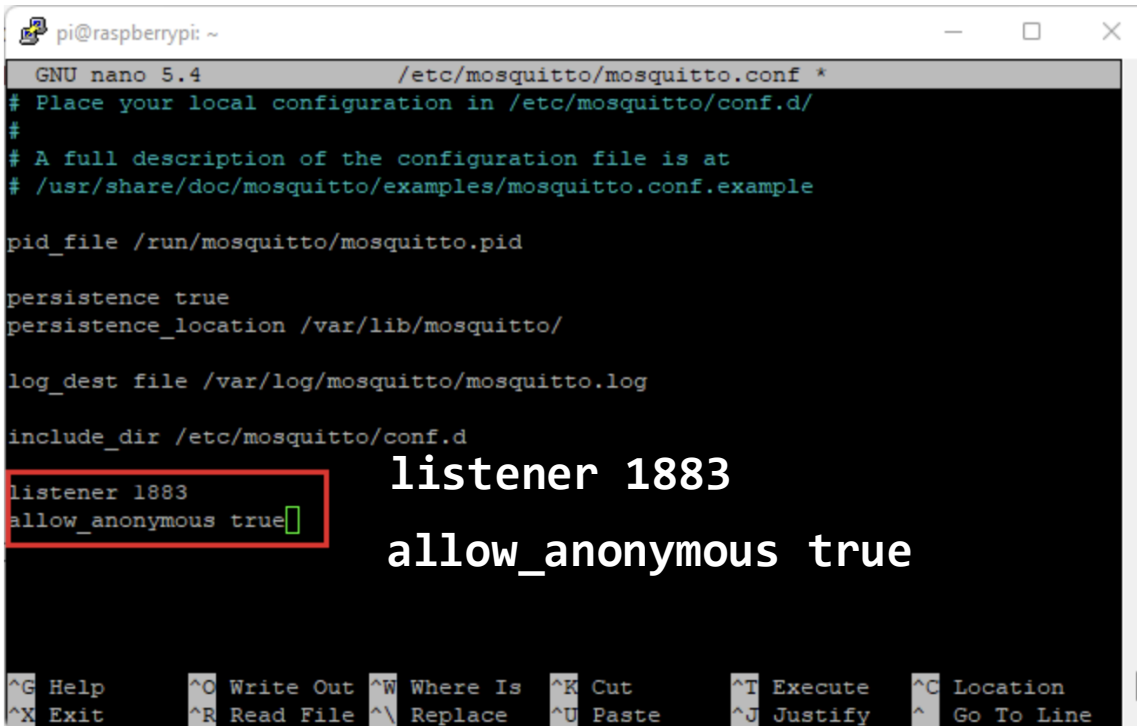
`hostname -I`

this is also OK to use



Step-4: Enable Remote Access to Mosquitto Broker (No Authentication)

sudo nano /etc/mosquitto/mosquitto.conf



```
pi@raspberrypi: ~
GNU nano 5.4 /etc/mosquitto/mosquitto.conf *
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example

pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto/

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d
listener 1883
allow_anonymous true

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

sudo systemctl restart mosquitto

sudo systemctl status mosquitto



Step-4: Publishing “Hello World!” Message to *testTopic* Topic

Install mosquitto-clients

sudo apt install -y mosquitto mosquitto-clients

Open a terminal window and type the following:

mosquitto_sub -d -t testTopic

```
pi@raspberrypi: ~
pi@raspberrypi:~ $ mosquitto_sub -d -t testTopic
Client (null) sending CONNECT
Client (null) received CONNACK (0)
Client (null) sending SUBSCRIBE (Mid: 1, Topic: testTopic, QoS: 0, Options: 0x00)
Client (null) received SUBACK
Subscribed (mid: 1): 0
```

mosquitto_sub -v -t '#' -h <IP address>

mosquitto_pub -d -t testTopic -m "Hello world!"

```
pi@raspberrypi: ~
pi@raspberrypi:~ $ mosquitto_sub -d -t testTopic
Client mosqsub/919-raspberrypi sending CONNECT
Client mosqsub/919-raspberrypi received CONNACK
Client mosqsub/919-raspberrypi sending SUBSCRIBE (Mid: 1, Topic: testTopic, QoS: 0)
Client mosqsub/919-raspberrypi received SUBACK
Subscribed (mid: 1): 0
Client mosqsub/919-raspberrypi received PUBLISH (d0, q0, r0, m0, 'testTopic', ... (12 bytes))
Hello world!
Client mosqsub/919-raspberrypi received PUBLISH (d0, q0, r0, m0, 'testTopic', ... (12 bytes))
Hello world!
```

```
pi@raspberrypi: ~
pi@raspberrypi:~ $ mosquitto_pub -d -t testTopic -m "Hello world!"
Client mosqpub/920-raspberrypi sending CONNECT
Client mosqpub/920-raspberrypi received CONNACK
Client mosqpub/920-raspberrypi sending PUBLISH (d0, q0, r0, m1, 'testTopic', ... (12 bytes))
Client mosqpub/920-raspberrypi sending DISCONNECT
pi@raspberrypi:~ $ mosquitto_pub -d -t testTopic -m "Hello world!"
Client mosqpub/922-raspberrypi sending CONNECT
Client mosqpub/922-raspberrypi received CONNACK
Client mosqpub/922-raspberrypi sending PUBLISH (d0, q0, r0, m1, 'testTopic', ... (12 bytes))
Client mosqpub/922-raspberrypi sending DISCONNECT
pi@raspberrypi:~ $
```

```
pi@raspberrypi: ~
pi@raspberrypi:~ $ mosquitto_sub -d -t testTopic
Client mosqsub/921-raspberrypi sending CONNECT
Client mosqsub/921-raspberrypi received CONNACK
Client mosqsub/921-raspberrypi sending SUBSCRIBE (Mid: 1, Topic: testTopic, QoS: 0)
Client mosqsub/921-raspberrypi received SUBACK
Subscribed (mid: 1): 0
Client mosqsub/921-raspberrypi received PUBLISH (d0, q0, r0, m0, 'testTopic', ... (12 bytes))
Hello world!
```



Types of MQTT messages

CONNECT — Is the client request to connect to the broker

CONNACK — Acknowledgement of the connect

PUBLISH — Publishes a message to a topic

PUBACK — Acknowledgement of the publish with QoS level 1

PUBREC — Acknowledgement of the publish with QoS level 2 (2nd packet)

PUBREL — Response to the PUBREC. (3rd packet when using QoS level 2)

PUBCOMP — Response to PUBREL (4th and last packet when using QoS lvl 2)

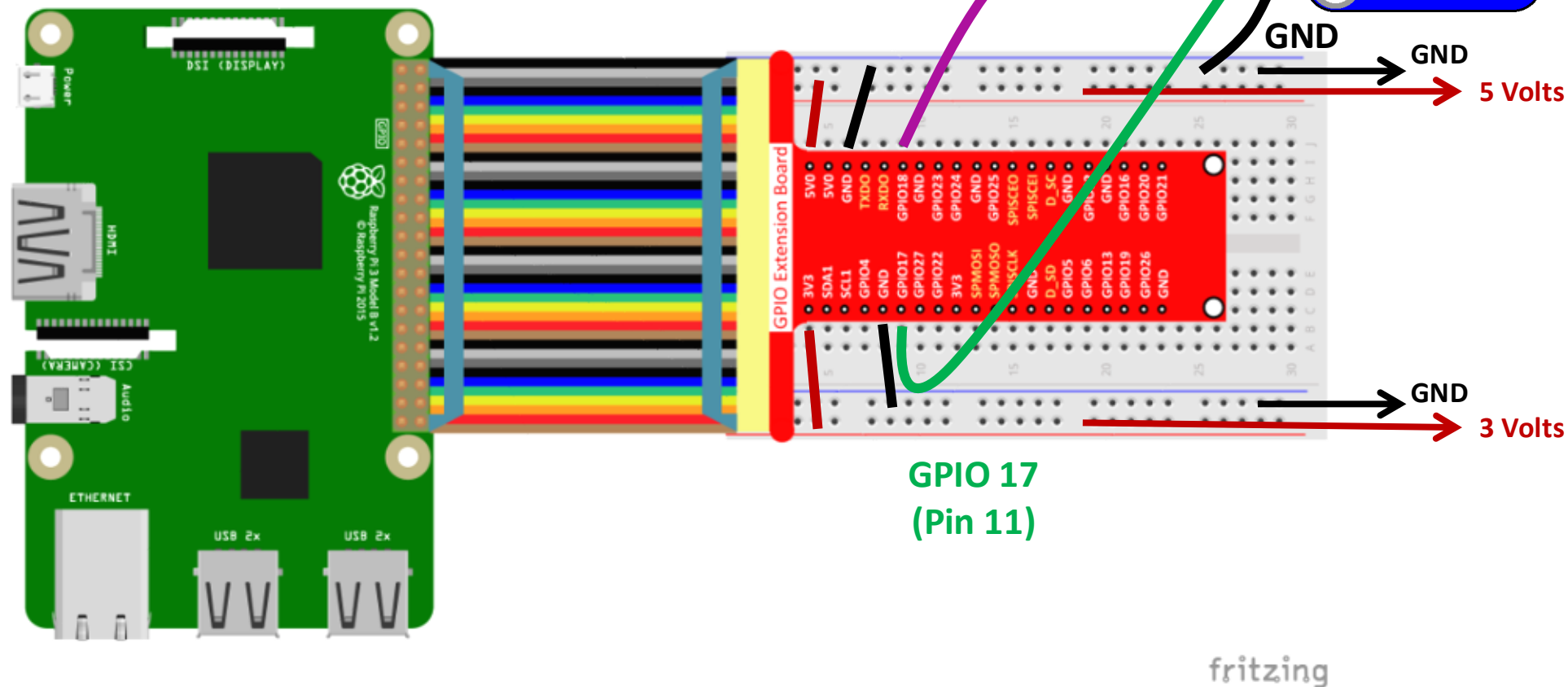
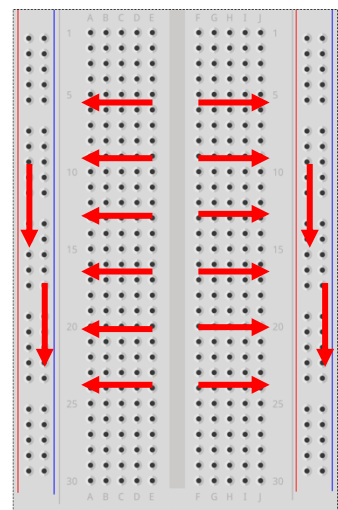
SUBSCRIBE — Packet from the client to subscribe to topics

SUBACK — Acknowledgement of the subscribe packet

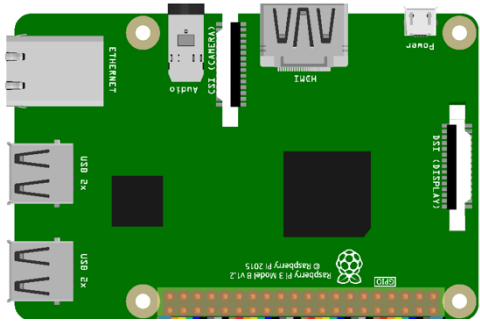
UNSUBSCRIBE — Packet from the client to unsubscribe from topics



Light up an LED with your Raspberry Pi 4 B



Another python
code to kick start
your Raspberry Pi
Model 3 B+ (RPi)



```
import RPi.GPIO as GPIO
import time
```

```
GPIO.setmode(GPIO.BOARD)
```

GPIO Extension Board			
1	• 3V3	• 5V0	2
3	• SDA1	• 5V0	4
5	• SCL1	• GND	6
7	• GPIO4	• TXD0	8
9	• GND	• RXD0	10
11	• GPIO17	• GPIO18	12
13	• GPIO27	• GND	14
15	• GPIO22	• GPIO23	16
17	• 3V3	• GPIO24	18
19	• SPMOSI	• GND	20
21	• SPMOSO	• GPIO25	22
23	• SPISCLK	• SPISCEO	24
25	• GND	• SPISCEI	26
27	• D_SD	• D_SC	28
29	• GPIO5	• GND	30
31	• GPIO6	• GPIO12	32
33	• GPIO13	• GND	34
35	• GPIO19	• GPIO16	36
37	• GPIO26	• GPIO20	38
39	• GND	• GPIO21	40

```
GPIO.setup(12, GPIO.OUT)
```

```
def loop():
    while True:
        GPIO.output(12, GPIO.HIGH)
        time.sleep(0.5)
        GPIO.output(12, GPIO.LOW)
        time.sleep(0.5)
```

```
def destroy():
    GPIO.output(12, GPIO.LOW)
    # Turn off all leds
    GPIO.cleanup()
```

```
if __name__ == "__main__":
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```



As a part of graded in-class lab
git clone MQTT codes from github repo and demonstrate
[10 points]



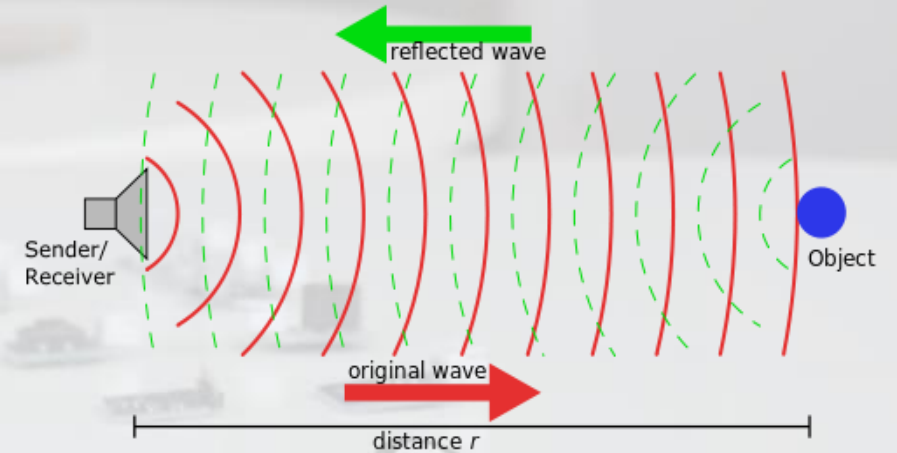
Ultrasound Signals and its Applications



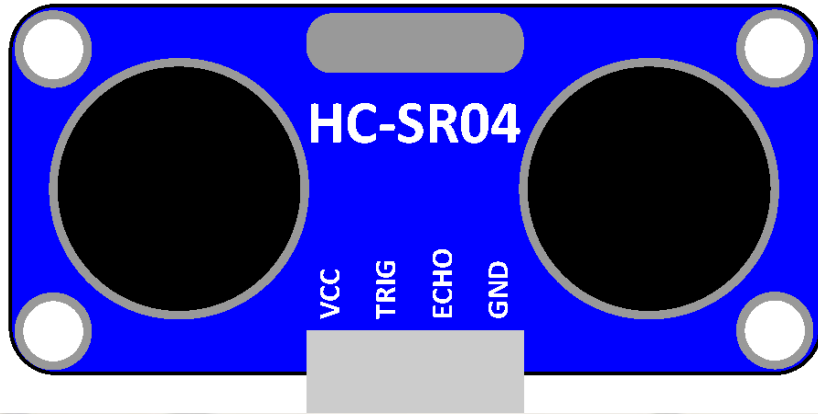
Source: <https://youtu.be/J-VVh5eqGA?si=G9iu055uV3MinyUA>

$$\text{Distance traversed} = (\text{Speed of sound}) \times (\text{Time elapsed}/2)$$

Icon Sources:
sensor by Carolina Cani; sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo:
<https://thenounproject.com/browse/icons/term/sensor/>
fire sensor by LAFS: <https://thenounproject.com/browse/icons/term/fire-sensor/>
Ultrasound by Shocho: <https://thenounproject.com/browse/icons/term/ultrasound/>
Network by Solikin; Network by Tippawan: <https://thenounproject.com/browse/icons/term/network/>
application by Chaowalit Koetchuea: <https://thenounproject.com/browse/icons/term/application/>



Know your Ultrasonic Sensor



The Ultrasonic sensor sends out ultrasonic waves to detect objects and measure distances.

Connectors:

4-pin connector wires

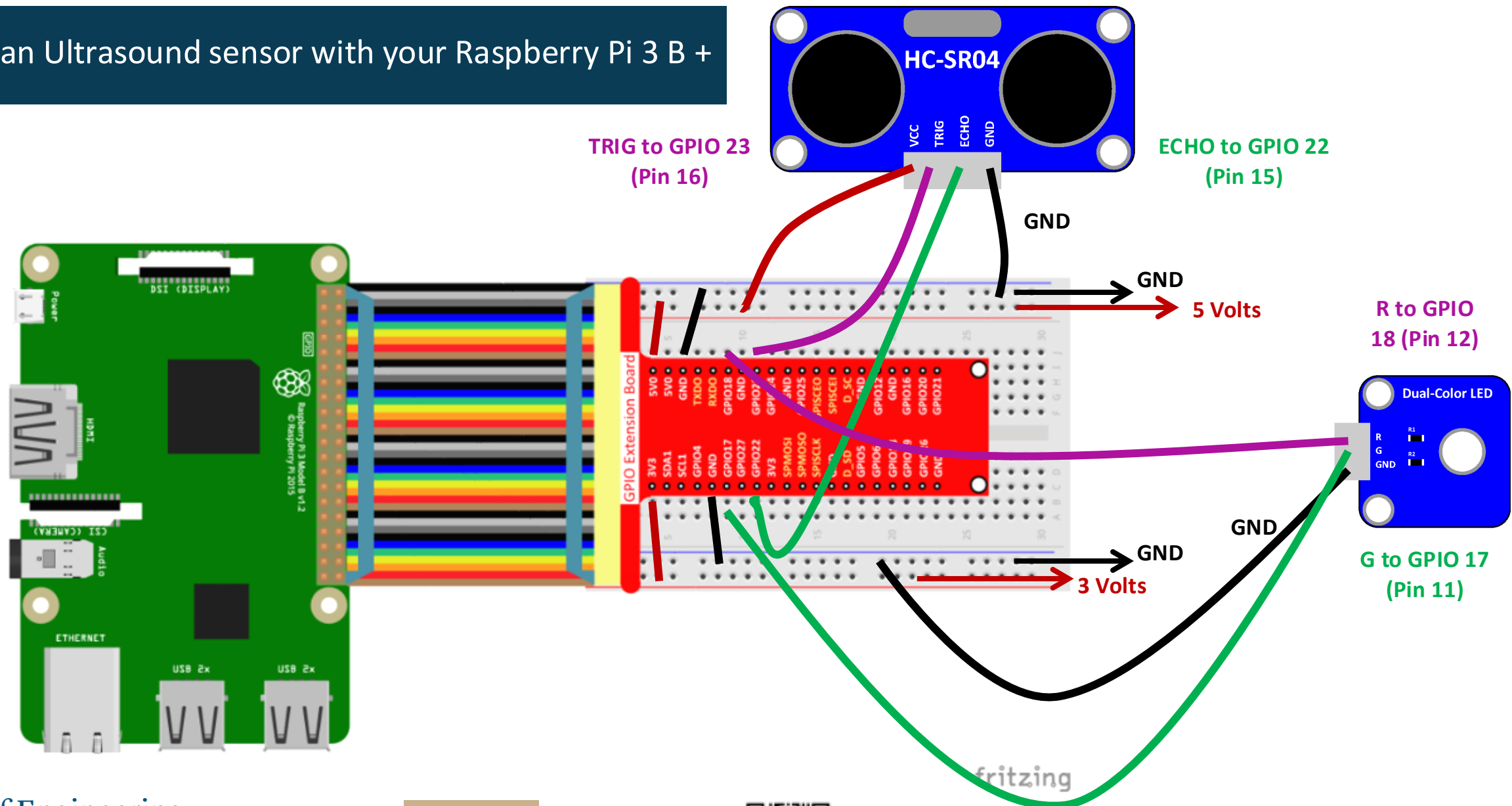
Or in some cases 5-pin connector wires

Goal of this HW segment:

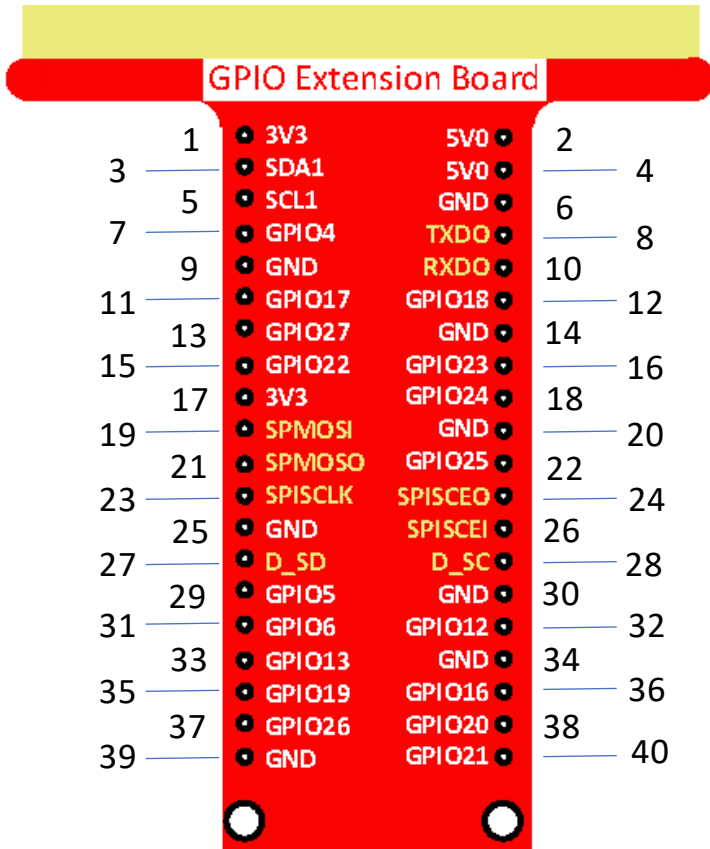
- **Co-work** (if you need to)
 - Observe, ask and try in groups
- **Make**
 - Build-a-hack
 - Ultrasound sensors and Raspberry Pi 4B boards
- **Analyze data using Python**



Start an Ultrasound sensor with your Raspberry Pi 3 B +



Pseudo-code to kick start your Raspberry Pi Model 3 B+ (RPi)



```
import LIBRARY as NAME
import ANOTHER_LIBRARY
```

INITIALIZE GPIO CHANNELS

DEFINE SETUP FUNCTION

```
GPIO.setmode(GPIO.BOARD)
GPIO.setup(CHANNEL-1, GPIO.OUT)
GPIO.setup(CHANNEL-2, GPIO.IN)
```

DEFINE DISTANCE FUNCTION

```
return (TIME_ELAPSED / 2) * 340 * 100
```

DEFINE LOOP FUNCTION

```
while True:
    ...
```

DEFINE DESTROY FUNCTION

CLEAN UP GPIO CHANNELS

```
if __name__ == "__main__":
    setup():
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```

User defined functions

Functions are blocks of reusable pieces of code

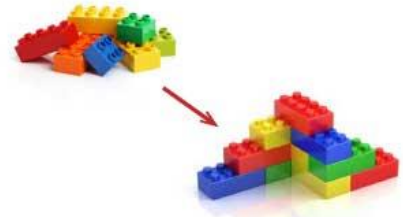
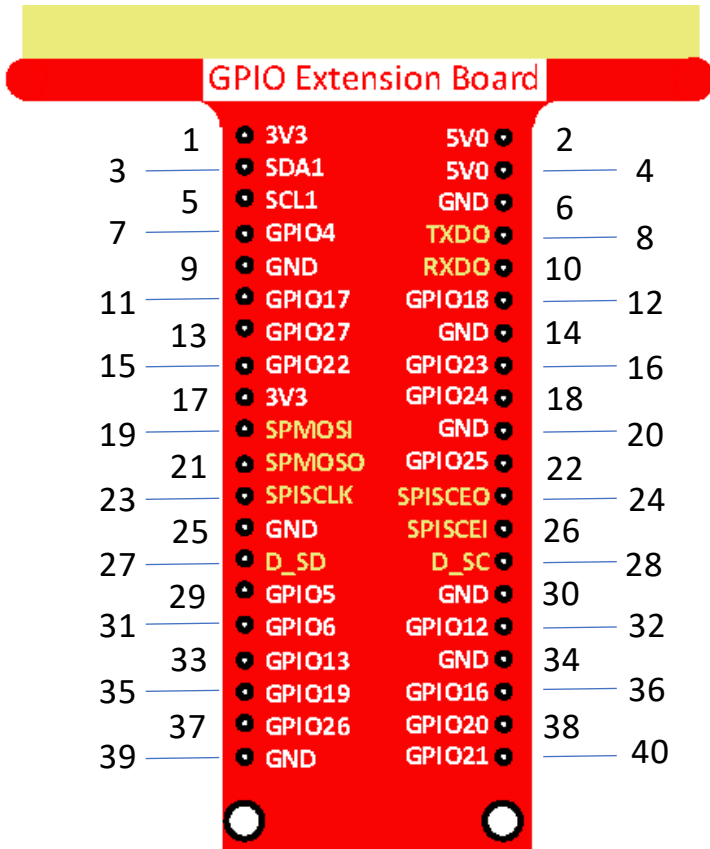


image source:
<https://www.teachmeanet.com/teaching-colors-to-preschoolers/lego-blocks-dip-art-activity-4-part/>
<https://www.123color.com/lego-color-activity-4-part/>

Entry point into the program – pulls in all user defined functions



A simple python code to kick start your Raspberry Pi Model 3 B+ (RPi)



```
import RPi.GPIO as GPIO
import time
```

```
TRIG = 16
ECHO = 15
def setup():
    GPIO.setmode(GPIO.BOARD)
    GPIO.setup(TRIG, GPIO.OUT)
    GPIO.setup(ECHO, GPIO.IN)
```

```
def distance():
    GPIO.output(TRIG, 0)
    time.sleep(0.000002)
    GPIO.output(TRIG, 1)
    time.sleep(0.00001)
    GPIO.output(TRIG, 0)

    while GPIO.input(ECHO) == 0:
        time1 = time.time()

    while GPIO.input(ECHO) == 1:
        time2 = time.time()

    during = time2 - time1
    return (during / 2) * 340 * 100
```

```
def loop():
    while True:
        dist = distance()
        print(dist, 'cm')
        print('')
        time.sleep(0.1)
```

```
def destroy():
    GPIO.cleanup()
```

```
if __name__ == "__main__":
    setup()
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```



Ultrasound sensor due on 02/25/2026 (10 points)

git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git

Goal-1

Set up a Python virtual environment
- “Sandbox” your Python project

Goal-2

Demonstrate your python script at boot
Using cron, the UNIX scheduler



Someone should summarize what we learned today

